T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	08/28/2026	**DATE SUBMITTED:**	09/02/2026
NAME:	Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

What procedures did you perform?

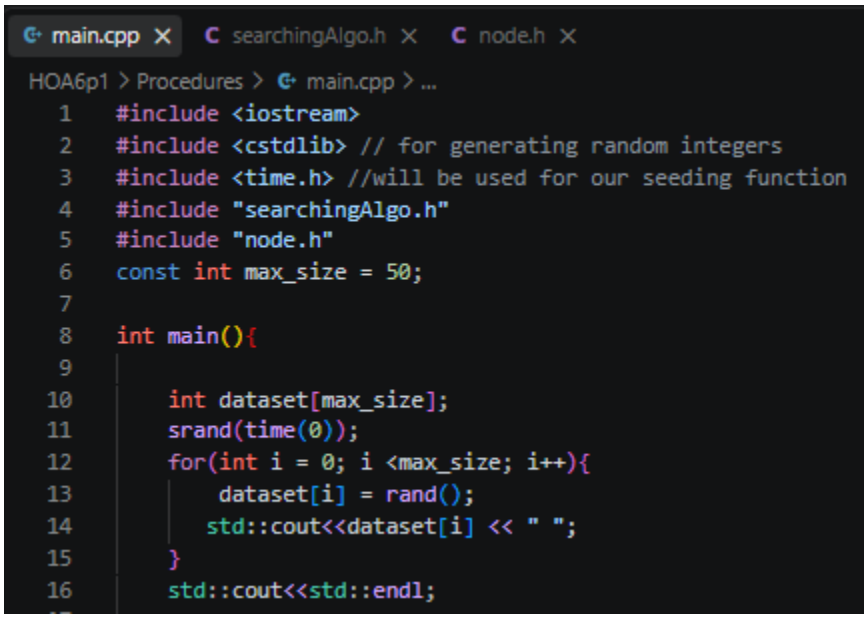
Screenshot	
Observations	First of all, from what I noticed after running the code it showed 50 random integers for the dataset. This was done by firstly having “cstdlib” which allows us to generate random integers. Next, a global declaration of “max_size” of 50 so we can just use it for the size of the array named “dataset.” After that a for loop for running the code 50 times to fill the array which line 13 is responsible for.

Table 6-1. Data Generated and Observations

Code

main.cpp searchingAlgo.h node.h

HOA6p1 > Procedures > main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib> // for generating random integers
#include <time.h> //will be used for our seeding function
#include "searchingAlgo.h"
#include "node.h"
const int max_size = 50;

int main(){

    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i <max_size; i++){
        dataset[i] = rand();
       std::cout<<dataset[i] << " ";
    }
    std::cout<<std::endl;

    std::cout<<"What number to find: ";
    int choice;
    std::cin >> choice;

    linearSearchAR(max_size, choice, dataset);
```

<table>
<tr><td></td><td>

main.cpp | searchingAlgo.h | node.h

HOA6p1 > Procedures > searchingAlgo.h > ...

```cpp
#include "node.h"

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template<typename T>
void linearSearchAR(int size, T dataFind, T array[]){
    int i = 0;
    //while loop implementation
    while(i < size){
        if(dataFind == array[i]){
            std::cout<<"Searching is successful\n";
            return;
        }
        i++;
    }
    std::cout<<"Searching is unsuccessful\n";

    //for loop implementation
    for(int j = 0; j < size; i++){
        if(dataFind == array[i]){
            std::cout<<"Searching is successful\n";
            return;
        }
    }
    std::cout<<"Searching is unsuccessful\n";

}
```

</td></tr>
<tr><td>Output</td><td>

```
PS C:\Users\USer\OneDrive\Documents\REALUBIT-CPE010-main\HOA6p1\Procedures> cd "c:\Users\USer\OneDrive\Documents\REALUBIT-CPE010-main\HOA6p1\Procedures\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
10564 15051 5602 29605 16089 12152 10199 23857 10797 25292 26153 28820 32567 18260 23808 19656 29400 18604 24455 15159 19361 27554 9428 18128 27797 15050 10635 18777 32048 24980 18335 25351 21160 32492 1690 4172 32370 2561 15936 20976 9736 23288 19567 7011 2026 11233 25
162 31570 20062 9706
What number to find: 9706
Searching is successful
PS C:\Users\USer\OneDrive\Documents\REALUBIT-CPE010-main\HOA6p1\Procedures>
```

</td></tr>
<tr><td>Observations</td><td>In this part of the code, we included the header file “searchingAlgo.h” for the logic of searching the array with linear searching. We have “int size, T dataFind, and T array[ ]” for the parameters so that when calling it in the “main.cpp” we can choose what we are searching for in the dataset. We use a while loop or for loop for outputting the condition whether the value we are searching for in the array is there, if not it will output as searching is unsuccessful.</td></tr>
</table>

Table 6-2a. Linear Search for Arrays

Code

```
//Creating a node for your name
Node<char> *char1 = new_node('J');
Node<char> *char2 = new_node('I');
Node<char> *char3 = new_node('M');
Node<char> *char4 = new_node('L');
Node<char> *char5 = new_node('O');
Node<char> *char6 = new_node('R');
Node<char> *char7 = new_node('D');

//Connect the nodes
char1->next = char2;
char2->next = char3;
char3->next = char4;
char4->next = char5;
char5->next = char6;
char6->next = char7;

linearSearchLL(char1, 'D');
```

```
template<typename T>
void linearSearchLL(Node<T> *nodes, T dataFind){
    //repeat the loop until the correct node is not NULL

    while(nodes != nullptr){

        //check if the item = current node data
        if(nodes->data == dataFind){
            std::cout<<"The data: "<< nodes->data<< " was Found successfully.\n";
            return;
        }
        // move on to the next node
        nodes = nodes->next;
    }
    //display unsuccessful
        std::cout<<"Searching is unsuccessful.\n";
}
```

Output

```
Searching is successful
PS C:\Users\USer\OneDrive\Documents\REALUBIT-
The data: D was Found successfully.
PS C:\Users\USer\OneDrive\Documents\REALUBIT-
```

Observations

We first make a linked list and point from one node to the next. For the header file we make a while loop of as long as the node does not point to nullptr it will search through the nodes one by one until what we are specifically looking for is found. Else, it would output as searching is unsuccessful.

Table 6-2b. Linear Search for Linked List

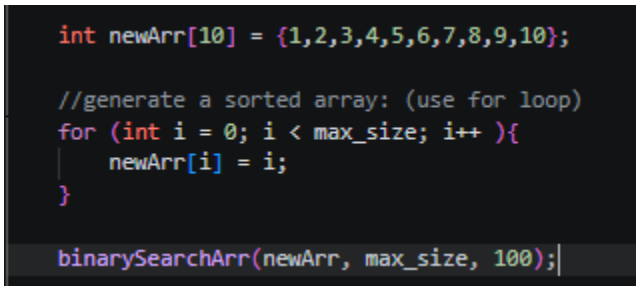
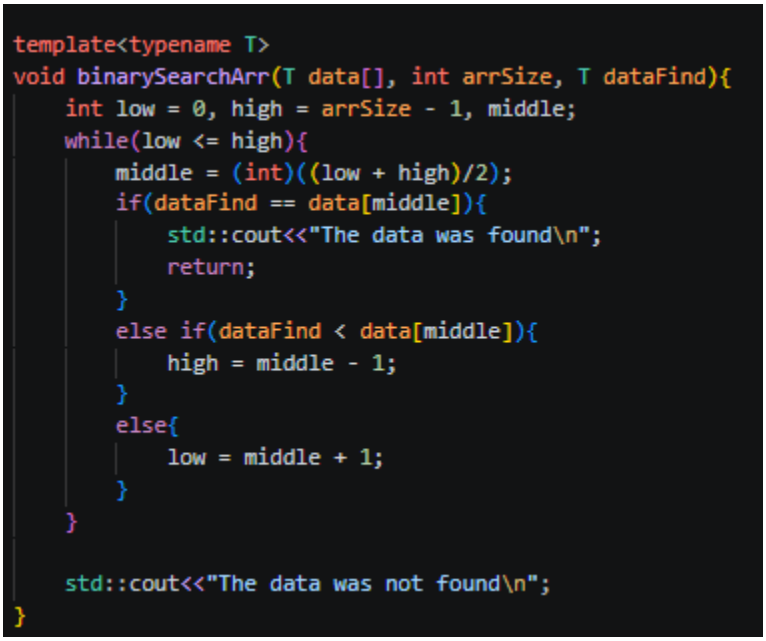
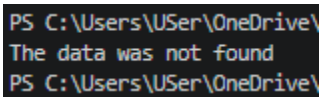
Code	 
Output	
Observations	From my observation, the data was not found because we only had the for loop run until 50. Apart from that we have the function of “binarySearchArr” with the parameters for finding what we need and the name and size of array itself. We first set high and low at the back and front of the array, before having the middle point as unappointed for the middle point.

Table 6-3a. Binary Search for Arrays

Code

```
Node<int> *char1 = new_node(10);
Node<int> *char2 = new_node(11);
Node<int> *char3 = new_node(12);
Node<int> *char4 = new_node(13);
Node<int> *char5 = new_node(14);
Node<int> *char6 = new_node(15);
Node<int> *char7 = new_node(16);

//Connect the nodes
char1->next = char2;
char2->next = char3;
char3->next = char4;
char4->next = char5;
char5->next = char6;
char6->next = char7;

binarySearchLL(char1, 18);
```

```
template<typename T>
void binarySearchLL(Node<T>* start, T dataFind){
    //step 1: Set start to the head of the list and set last to null
    Node<T>* last = nullptr;

    // Middle element is calculated using the two pointers approach discussed above
    while(start != last){
        Node<T> *middle = getMiddle(start, last);

        //If the middle element is same as the key to be sorted, we return it
        if(middle->data == dataFind){
            std::cout<<"searching is Successfull\n";
            return;
        }
        else if(middle->data > dataFind){
            last = middle;
        }
        else{
            start = middle->next;
        }
    }

    std::cout<<"Searching is unsuccessful\n";
}
#endif
```

Output

```
PS C:\Users\USer\OneDrive\Docu
Searching is unsuccessful
PS C:\Users\USer\OneDrive\Docu
```

Observations

This time, instead of char we use int since we are going to use integers for the binary search with linked list implementation. Inside the function we have the parameter “Node<T>* start”

	which starts looking at the start of the linked list and the value or integer of what we are actually looking for inside the linked list. Next is a while loop that will keep running until the start and last meets, inside the while loop is for getting the middle of that linked list. We use an if-else statement for the condition of finding the middle element and outputting a “searching is successful” if that value is found before returning. Otherwise, it would just run if the searching is unsuccessful.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PROBLEM 1

nodes.h | searchCore.h | main.cpp

HOA6p1 > Supplementary > nodes.h > Node<T>

```
#ifndef NODES_H
#define NODES_H

template<typename T>
class Node{
public:
    T data;
    Node *next = nullptr;
    Node *prev = nullptr;

};

template<typename T>
Node<T> *new_node(T newData){
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;

}

#endif
```

Figure 1. Code Snippet of nodes.h

nodes.h | searchCore.h | main.cpp

HOA6p1 > Supplementary > searchCore.h > linearSearchAR<T>(int, T, T [])

```cpp
#ifndef SEARCH_CORE_H
#define SEARCH_CORE_H
#include "nodes.h"

template<typename T>
int linearSearchAR(int size, T dataFind, T array[]){

    int counter = 0;

    for(int j = 0; j < size; j++){
        counter++;
        if(dataFind == array[j]){
            std::cout<<"Searching is successful\n";
            std::cout<<"Counter: "<<counter<<"\n";
            return counter;
        }

    }
    std::cout<<"Searching is unsuccessful\n";
    std::cout<<"Counter: "<<counter<<"\n";
    return counter;

}

template<typename T>
int linearSearchLL(Node<T> *head, T dataFind){
    //repeat the loop until the correct node is not NULL
    int counter = 0;

    while(head != nullptr){
        counter++;
        //check if the item = current node data
        if(head->data == dataFind){
            std::cout<<"The data: "<< head->data<< " was Found successfully.\n";
            std::cout<<"Counter: "<<counter<<"\n";
            return counter;
        }
        // move on to the next node
        head = head->next;
    }
    //display unsuccessful
        std::cout<<"Searching is unsuccessful.\n";
    std::cout<<"Counter: "<<counter<<"\n";
    return counter;
}



#endif //SEARCH_CORE_H
```

Figure 2. Code Snippet of searchCore.h

nodes.h | searchCore.h | main.cpp

HOA6p1 > Supplementary > main.cpp > insertAtEnd<T>(Node<T>**, T)

```cpp
#include <iostream>
#include "nodes.h"
#include "searchCore.h"

template<typename T>
void insertAtEnd(Node<T> **head, T newData){
    Node<T> *newNode = new_node(newData);
    Node<T> *last = *head;

    if(*head == nullptr){
        *head = newNode;
        return;
    }

    while(last->next != nullptr){
        last = last->next;
    }

    last->next = newNode;
    newNode->prev = last;
}

int main(){
    const int size = 10;
    int Data[size] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int searchKey = 18;

    std::cout << "Linear Search in Array:\n";
    linearSearchAR(size, searchKey, Data);

    std::cout << "\nLinear Search in Linked List:\n";
    Node<int> *head = nullptr;
    for (int i = 0; i < size; i++) {
        insertAtEnd(&head, Data[i]);
    }

    linearSearchLL(head, searchKey);

    Node<int> *current = head;
    while (current != nullptr) {
        Node<int> *nextNode = current->next;
        delete current;
        current = nextNode;
    }

    return 0;
}
```

Figure 3. Code Snippet of main.cpp

```
Linear Search in Array:
Searching is successful
Counter: 2

Linear Search in Linked List:
The data: 18 was Found successfully.
Counter: 2
```

Figure 4. Snippet of the output

Answer to the Question: It would need 2 comparisons so that the key is found since the placement of 18 in the list or array is on the second position.

PROBLEM 2

nodes.h | searchCore.h | main.cpp

HOA6p1 > Supplementary > searchCore.h > linearSearchLL<T>(Node<T>*, T)

```cpp
#ifndef SEARCH_CORE_H
#define SEARCH_CORE_H
#include "nodes.h"

template<typename T>
int linearSearchAR(int size, T dataFind, T array[]){

    int comparisons = 0;
    int matches = 0;

    for(int j = 0; j < size; j++){
        comparisons++;
        if(dataFind == array[j]){
            matches++;
        }

    }
    if(matches > 0){
        std::cout<<"Searching is successful\n";
    } else {
        std::cout<<"Searching is unsuccessful\n";
    }
    std::cout<<"Found comparisons: "<<comparisons<<"\n";
    std::cout<<"Found matches: "<<matches<<"\n";
    return matches;

}

template<typename T>
int linearSearchLL(Node<T> *head, T dataFind){
    // Traverse the entire list and count every matching node.
    int comparisons = 0;
    int matches = 0;

    while(head != nullptr){
        comparisons++;
        if(head->data == dataFind){
            matches++;
        }
        // move on to the next node
        head = head->next;
    }
    if(matches > 0){
        std::cout<<"The data: "<< dataFind<< " was Found successfully.\n";
    } else {
        std::cout<<"Searching is unsuccessful.\n";
    }
    std::cout<<"Found comparisons: "<<comparisons<<"\n";
    std::cout<<"Found matches: "<<matches<<"\n";
    return matches;
}

#endif //SEARCH_CORE_H
```

Figure 5. Code Snippet of the Modified linearSearchArr and linearSearchLL

```
Linear Search in Array:
Searching is successful
Found comparisons: 10
Found matches: 2

Linear Search in Linked List:
The data: 18 was Found successfully.
Found comparisons: 10
Found matches: 2
```

Figure 6. Snippet of the Output in the Terminal

PROBLEM 3

```cpp
template<typename T>
int binarySearchArr(const T data[], int arrSize, T dataFind){
    int low = 0;
    int high = arrSize - 1;
    int iterations = 0;

    while (low <= high){
        int middle = low + (high - low) / 2;
        iterations++;

        std::cout<< "Iteration: "<<iterations<<"\n";
        std::cout<< "Low: "<<low<<"\n";
        std::cout<< "High: "<<high<<"\n";

        if (data[middle] == dataFind){
            std::cout<<"The data was found\n";
            return iterations;
        }
        else if (data[middle] < dataFind){
            low = middle + 1;
        }
        else{
            high = middle - 1;
        }
    }

    std::cout<<"The data was not found\n";
    return iterations;
}
```

Figure 7. Code Snippet of binarySearchArr in the header file

```
int result = -1;

int Data2[size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int searchKey2 = 8;

std::cout << "\nBinary Search in Array:\n";
binarySearchArr(Data2, size, searchKey2);

if (result != -1) {
    std::cout << "The data: " << searchKey2 << " was found at index: " << result << "\n";

}
return 0;
```

Figure 8. Code Snippet of testing the function against the lis

```
Binary Search in Array:
Iteration: 1
Low: 0
High: 9
Iteration: 2
Low: 0
High: 3
Iteration: 3
Low: 2
High: 3
Iteration: 4
Low: 3
High: 3
The data was found
```

Figure 9. Snippet of the output in the terminal

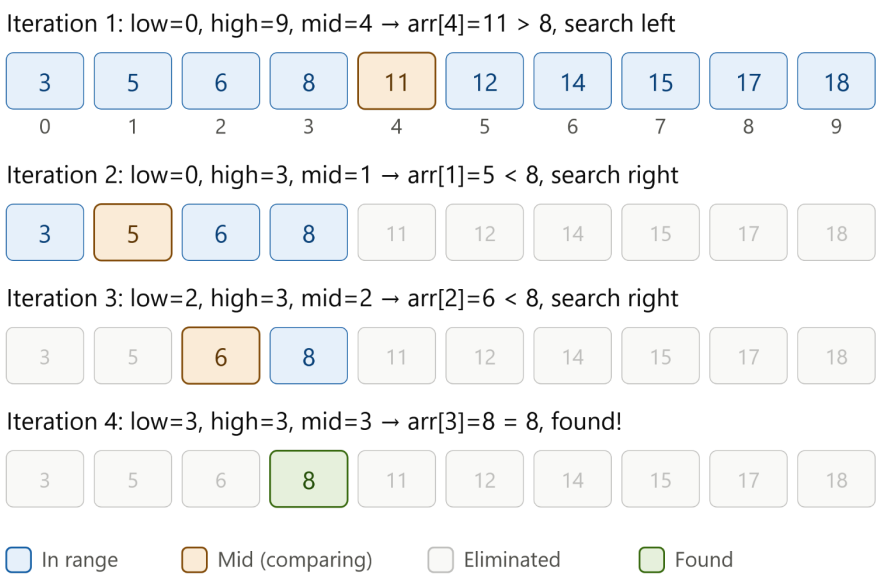


Figure 10. Snippet of the Drawing

Algorithm:
Step 1: low and high starts as both ends
Step 2: Looping condition of low is less than or equal to high (low <= high)
Step 3: middle = low (high - low) / 2

PROBLEM 4

```cpp
template<typename T>
int binarySearchRecursive(const T data[], int low, int high, T dataFind, int &iterations){
    if (low > high){
        std::cout<<"The data was not found\n";
        return iterations;
    }

    int middle = low + (high - low) / 2;
    iterations++;

    std::cout<< "Iteration: "<<iterations<<"\n";
    std::cout<< "Low: "<<low<<"\n";
    std::cout<< "High: "<<high<<"\n";

    if (data[middle] == dataFind){
        std::cout<<"The data was found\n";
        return iterations;
    }
    else if (data[middle] < dataFind){
        return binarySearchRecursive(data, middle + 1, high, dataFind, iterations);
    }
    else{
        return binarySearchRecursive(data, low, middle - 1, dataFind, iterations);
    }
}
```

Figure 11. Code Snippet of the function binarySearchRecursive

```cpp
int iterations = 0;

std::cout <<"\nBinary Search Recursive in Array:\n";
binarySearchRecursive(Data2, 0, size - 1, searchKey2, iterations);
if (result != -1) {
    std::cout << "The data: " << searchKey2 << " was found at index: " << result << "\n";
}
```

Figure 12. Code Snippet of testing binarySearchRecursive using the 2nd sorted list

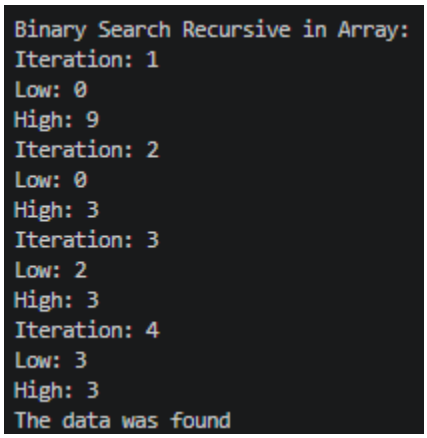


Figure 13. Snippet of the Output in the terminal

Conclusion

In this Hands-on Activity, I learned the different implementations of binary and linear searching. I also learned the equation of finding (low + high)/2 which allows us to find the middle of the low and high.

Following that, the procedure is that starting with the linear search using array and linked list implementation which using for loops proved to be much more efficient that while loops for the conditioning. Where a random data set was made and if we choose one of the integers inside that dataset it would prove to be found, otherwise the searching would be unsuccessful. Then for the binary search by dividing to the half we can slowly reduce the list.

After that is the supplementary activity where I adapted the code from the original header file and only changed and added some variables. Since we needed the counter at first I added the variable counter that would run for each time index it passes through. Followed by the comparison and matches where it followed the same logic but add a new variable to count the matches since there are two keys (18) in the given list. Then the problem 3 and 4 where we implement an array using binary search to get the middle and slowly cut it off until the data we are looking for (key) is found.

Overall the activity itself was still difficult but only required minor changes to implement the algorithm which allowed me to do the code. It was still difficult, especially since I would need to practice basing off from algorithms from now on rather than being given the actual code itself.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly AI, Claude AI**

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction and Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Grammarly only after I was able to make the whole paragraph then start to fix it using grammarly which can essentially translate that it did not replace my original thinking. Then the explanation of concepts was mainly used to explain the recursive like how does it work and not the actual code itself.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.